Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Operating System

Study Material

Ms. RASHMI PAL
CSE, PIET
Parul University

CHAPTER 1 – INTRODUCTION TO OPERATING SYSTEMS

1.1 What is an Operating System?

An **Operating System (OS)** is a system software that acts as an **interface between user and hardware**. It manages hardware resources and provides an environment for application programs to run.

Key Roles of OS

- Manages hardware (CPU, Memory, I/O devices, Storage)
- Provides user interface
- Executes and manages programs
- Ensures security and protection
- Efficient resource utilization

Simple Definition:

👉 *An OS controls hardware and allows user applications to run smoothly.*

1.2 Goals of an Operating System

Primary Goals

1. **Convenience** — Makes the computer easier to use
2. **Efficiency** — Maximizes performance and resource utilization
3. **Security** — Protects data and users
4. **Reliability** — Ensures stable system execution

Secondary Goals

- Scalability
- Robustness
- Portability
- Minimizing response time

1.3 Types of Operating Systems

1. Batch Operating System

- Executes jobs without user interaction
- Jobs collected → Processed in batches
- Used in mainframes
  Example: Early IBM mainframe OS

2. Time-Sharing (Multitasking) OS

- Multiple users use the system at the same time
- CPU switches rapidly between tasks
  Example: UNIX, Linux

3. Distributed Operating System

- Multiple computers connected to work as one
  Example: LOCUS, Amoeba

4. Network Operating System

- Provides network-related services
  Example: Windows Server, Novell NetWare

5. Real-Time Operating System (RTOS)

- Output must be within strict timing constraints
- Used in embedded systems
  Example: VxWorks, QNX, RTLinux

6. Mobile Operating Systems

Example: Android, iOS

1.4 Operating System Services

The OS provides essential services to applications and users:

1. Program Execution

Loads program → Executes → Terminates safely

2. I/O Operations

Interaction with keyboard, mouse, printer, disk etc.

3. File-System Manipulation

Creating, reading, writing, deleting files.

4. Communication Services

Between processes using:

- Message passing
- Shared memory

5. Error Detection

Monitors hardware/software for errors.

6. Resource Allocation

CPU scheduling, memory allocation, I/O assignment.

7. Protection & Security

Prevents unauthorized access, manages permissions.

1.5 System Calls

System calls provide the interface between **user programs** and the **OS kernel**.

Types of System Calls

1. **Process Control**
   - create(), exit(), wait()
2. **File Management**
   - open(), read(), write(), close()
3. **Device Management**
   - ioctl(), request device
4. **Information Maintenance**
   - getpid(), gettime()
5. **Communication**

- send(), receive()

Example (Linux):

```
fd = open("file.txt", O_RDONLY);
```

1.6 Structure of an Operating System

Different OSes use different architectures:

1. Monolithic System

- Entire OS runs as a single program
- Fast but hard to maintain
  Example: Early UNIX, MS-DOS

2. Layered Architecture

- OS divided into layers
- Each layer depends on lower layers
  Example: THE system by Dijkstra

3. Microkernel Architecture

- Only essential components in kernel
- Other services run as user processes
  Example: QNX, Minix

4. Modules

- Object-oriented approach
- Loadable kernel modules
  Example: Modern Linux

1.7 Virtual Machines

A Virtual Machine (VM) simulates a complete physical computer system.

Types

- **System VM** – Runs entire OS (Example: VMware, VirtualBox)
- **Process VM** – Runs a single program (Example: JVM)

Benefits

- Isolation
- Security
- Portable development environment
- Efficient resource utilization

1.8 Generations of Operating Systems

1st Generation (1940–50s): Vacuum Tubes

- No OS
- Manual machine code programming

2nd Generation (1950–60s): Batch Systems

- Job scheduling
- Batch OS introduced

3rd Generation (1960–80s): Multiprogramming

- Time-sharing

- Mainframes

4th Generation (1980–Present): Personal Computers

- GUI-based OS (Windows, macOS)

5th Generation: Intelligent OS / Cloud OS

- Virtualization
- Distributed OS
- Cloud and mobile systems

1.9 Summary (For Quick Revision)

- OS = Interface between user & hardware
- Types include: Batch, Time-sharing, RTOS, Distributed
- OS provides services like memory management, file handling, CPU scheduling
- System calls allow user programs to interact with the OS
- Structures: Monolithic, Layered, Microkernel
- Virtual machines abstract hardware and allow multiple OS instances

CHAPTER 2 – PROCESSES, THREADS & CPU SCHEDULING

2.1 What is a Process?

A **Process** is a program in execution.
It includes:

- Program code (text section)
- Data section
- Stack (function calls, parameters)
- Heap (dynamic memory)
- Program counter and registers

Characteristics of a Process

- Dynamic (changes state)
- Independent entity
- Requires resources (CPU time, memory, files)

2.2 Process States

A process changes states while executing:

Basic Process States

1. **New** – Process created
2. **Ready** – Waiting for CPU
3. **Running** – CPU executing process
4. **Waiting/Blocked** – Waiting for I/O or event
5. **Terminated** – Finished execution

State Transition Diagram

```
NEW → READY → RUNNING → WAITING → READY → TERMINATED
```

2.3 Process Control Block (PCB)

PCB is a data structure that stores all information about a process.

Contents of PCB

- Process ID (PID)
- Process state
- Program counter
- CPU registers
- Scheduling information
- Memory-management info
- I/O status information
- Accounting information

👉 *The OS uses PCB to manage and switch processes.*

2.4 Context Switching

Context switching is the process of saving CPU state of the running process and loading CPU state of another process.

When does it occur?

- Interrupts
- Time slice expires
- I/O completion
- System calls

👉 Frequent context switching reduces CPU efficiency.

2.5 Threads

A **Thread** is the smallest unit of CPU execution.

Process vs Thread

Process	Thread
Independent execution	Part of a process
Own memory	Shares memory
Heavyweight	Lightweight
Slow creation	Fast creation

Types of Threads

- **User-level threads**
- **Kernel-level threads**
- **Hybrid models**

2.6 Multithreading Models

Many-to-One

- Many user threads map to one kernel thread
- Not scalable

One-to-One

- Each user thread → One kernel thread
- More concurrency
- Example: Windows, Linux

Many-to-Many

- Many user threads → Many kernel threads
- Best flexibility

2.7 Process Scheduling

Scheduling decides which process runs next on CPU.

Goals of CPU Scheduling

- Maximize CPU utilization
- Maximize throughput
- Minimize turnaround time
- Minimize waiting time
- Minimize response time

2.8 Types of Schedulers

1. Long-Term Scheduler (Job Scheduler)

- Chooses processes to enter ready queue
- Controls degree of multiprogramming

2. Short-Term Scheduler (CPU Scheduler)

- Selects process for CPU
- Very fast decision-making

3. Medium-Term Scheduler

- Swaps processes in/out of memory
- Handles CPU load balancing

2.9 Scheduling Criteria

- **CPU Utilization** – Keep CPU as busy as possible
- **Throughput** – Number of processes executed per unit time
- **Turnaround Time** – Total time taken to complete a process
- **Waiting Time** – Time spent in ready queue
- **Response Time** – Time to produce first output

2.10 CPU Scheduling Algorithms

---

1. First-Come First-Served (FCFS)

- Non-preemptive
- Served in order of arrival

Advantages: Simple
Disadvantages: Long waiting time (convoy effect)

---

2. Shortest Job First (SJF)

- Non-preemptive or preemptive
- Process with shortest burst time executes next

Advantages: Minimum average waiting time
Disadvantages: Hard to know job length

---

3. Round Robin (RR)

- Time quantum assigned
- Preemptive
- Fair for all users

Best for: Time-sharing systems

---

4. Priority Scheduling

- Highest priority executes first
- Can be preemptive or non-preemptive

Problem: Starvation
Solution: Aging

---

5. Multi-Level Queue Scheduling

- Ready queue divided into multiple queues (foreground, background)
- Different scheduling for each queue

---

6. Multi-Level Feedback Queue (MLFQ)

- Processes can move between queues
- Most flexible system

2.11 Multicore and Multiprocessor Scheduling

Modern CPUs have multiple cores.
Scheduling must balance load across cores.

Models

- Asymmetric multiprocessing (ASMP)
- Symmetric multiprocessing (SMP)

2.12 Summary of Chapter 2

- Process = program in execution
- PCB stores process information
- Context switching enables multitasking
- Threads provide lightweight execution
- Scheduling determines CPU allocation
- Algorithms include FCFS, SJF, RR, Priority, MLFQ
- Multicore scheduling important in modern systems

Chapter 3 – Inter-Process Communication (IPC)

1. Introduction to IPC

Inter-Process Communication (IPC) refers to mechanisms that allow processes to **exchange data**, **synchronize actions**, and **coordinate execution**.

Why IPC is needed?

- Processes often run **concurrently**.
- They may need to share resources or data.
- Without coordination → **race conditions**, inconsistent results.

---

2. Critical Section Problem

A *critical section* is a code segment where a process accesses shared variables or resources.

Requirements of a good Critical Section solution

1. **Mutual Exclusion** – Only one process in the critical section at a time.
2. **Progress** – A process outside CS must not block others unnecessarily.
3. **Bounded Waiting** – No process should wait forever.

---

3. Race Conditions

- Arises when multiple processes access shared data simultaneously.
- Final output depends on execution order → **unpredictable**.
- Example: Two processes incrementing a shared counter.

---

4. Mutual Exclusion

To prevent race conditions, we enforce **mutual exclusion** using:

- Hardware methods
- Software algorithms
- Synchronization tools

---

5. Hardware Solutions

5.1 Strict Alternation

- Forces processes to strictly take turns.
- Ensures mutual exclusion but **violates progress**.

5.2 Peterson's Solution

A classical software-based solution for two processes.
Uses:

- `flag[i]` → intent
- `turn` → whose turn

Satisfies all three: **mutual exclusion**, **progress**, **bounded waiting**.

---

6. The Producer–Consumer Problem

A classical IPC synchronization problem.

Scenario:

- **Producer** generates data items and puts them in a buffer.
- **Consumer** removes items from the buffer.

Challenges:

- Buffer overflow/underflow control.
- Proper synchronization so consumer does not consume before production.

---

7. Semaphores

One of the most widely used synchronization tools.

Two types:

- **Binary Semaphore** (0 or 1)
- **Counting Semaphore** (integer value)

Operations:

- `wait()` / `P()` – decrement or block
- `signal()` / `V()` – increment or wake a process

Used to solve:

- Mutual exclusion
- Producer–Consumer
- Readers–Writers

---

8. Event Counters

- Synchronization mechanism using counters to represent events.
- Helps maintain ordering between processes.

---

9. Monitors

A high-level synchronization construct.

Features:

- Combines **data** + **procedures** + **mutual exclusion**.
- Only one process can execute a monitor procedure at a time.
- Uses *condition variables* (`wait`, `signal`).

---

10. Message Passing

When shared memory is not available, IPC occurs via message exchange.

Models:

- **Direct communication** – processes name each other.
- **Indirect communication** – uses **mailboxes/ports**.

Operations:

- `send(message)`
- `receive(message)`

Suitable for:

- Distributed systems
- Systems with no shared memory

---

11. Classical IPC Problems

11.1 Readers–Writers Problem

- Readers read a shared resource simultaneously.
- Writers require exclusive access.
- Goal: Ensure correct synchronization.

11.2 Dining Philosopher Problem

- Five philosophers sharing forks.
- Challenge: Avoid deadlock & starvation.

11.3 Producer–Consumer Problem

(Already discussed)

---

12. Summary of IPC Mechanisms

Mechanism	Shared Memory?	Synchronization Method	Usage
Hardware Locking	Yes	Atomic instructions	Low-level sync
Semaphores	Yes	wait/signal	General-purpose sync
Monitors	Yes	High-level constructs	Modern languages
Message Passing	No	Send/Receive	Distributed systems

Mechanism	Shared Memory?	Synchronization Method	Usage

Chapter 4 – Deadlocks

1. Introduction to Deadlocks

A **deadlock** is a situation where a group of processes are permanently blocked because each process is holding a resource and waiting for another resource held by another process.

Real-life analogy:
Two people in a narrow corridor trying to pass each other but both blocking the way.

---

2. Definition of Deadlock

A system is in deadlock when the following conditions hold:

- Each process in the set is waiting for an event that only another process in the set can cause.
- None of the processes can proceed → system is *stuck*.

Deadlocks occur in systems where resources must be **mutually exclusive** and processes request them at runtime.

---

3. Necessary and Sufficient Conditions for Deadlock

A deadlock occurs *only if all four* of these conditions hold simultaneously:

1. Mutual Exclusion

- Resources cannot be shared.
- Example: printer, tape drive.

2. Hold and Wait

- A process is holding at least one resource and requests additional resources.

3. No Preemption

- Resources cannot be forcibly taken away; they must be released voluntarily.

4. Circular Wait

- A cycle of processes exists, each waiting for a resource held by the next.

Deadlock exists ⇔ all four conditions hold.

---

4. Deadlock Handling Strategies

There are four major strategies:

1. **Deadlock Prevention**
2. **Deadlock Avoidance**

3. **Deadlock Detection**
4. **Deadlock Recovery**

---

5. Deadlock Prevention

This method ensures that *at least one* of the four necessary conditions **never** occurs.

Ways to prevent deadlock:

1. Prevent Mutual Exclusion

- Make resources sharable (not always possible).

2. Prevent Hold and Wait

- Require processes to request all resources at once.
  Disadvantage: Low resource utilization.

3. Prevent No Preemption

- If a process is denied a resource, it releases all current resources.

4. Prevent Circular Wait

- Impose a **total order** on resource types.
- Processes must request resources in increasing order.

---

6. Deadlock Avoidance

Allows the system to enter only *safe states*.

Key idea:

Before allocating a resource, check whether doing so leads to a potential deadlock.

Banker's Algorithm

(Explicitly listed in your syllabus
Subject_Syllabus_303105251 (1)
)

Used to avoid deadlocks by:

- Checking if the system will remain in a **safe state** after resource allocation.
- If safe → grant request.
- If unsafe → do not grant request.

Components:

- **Available:** Resources currently available.
- **Max:** Maximum demand of each process.
- **Allocation:** Resources currently allocated.
- **Need:** `Max − Allocation`.

The algorithm simulates allocation to verify future safety.

---

7. Deadlock Detection

Used when the system does **not prevent or avoid** deadlocks.

Deadlock Detection Algorithm

- Periodically checks for cycles in the **Resource-Allocation Graph**.
- If a cycle is found → deadlock exists.

Complexity:

- Typically $O(n^2)$ for graph-based detection.

---

8. Deadlock Recovery

Once a deadlock is detected, the OS must recover.

Methods:

1. Process Termination

- Abort **one process at a time** until deadlock is removed.
- Or abort **all processes** in the cycle.

2. Resource Preemption

- Take resources away from some processes.
- Challenges:
  - Rollback required.
  - Selecting the victim (which process loses its resources).
  - Avoid starvation.

---

9. Comparison of Strategies

Strategy	Pros	Cons
Prevention	Simple; guarantees no deadlock	Inefficient; low resource utilization
Avoidance	More flexible; uses Banker's algorithm	Requires prior knowledge; overhead
Detection	Works with dynamic resource requests	Must handle recovery
Recovery	Gets system back to normal	May cause rollbacks or process loss

---

10. Summary

- Deadlocks severely affect system performance.
- Four conditions must simultaneously hold for deadlocks.
- OS can **prevent**, **avoid**, **detect**, or **recover** from deadlocks.
- Banker's algorithm is key for avoidance.
- Detection requires cycle finding.
- Recovery relies on preemption or termination.

Chapter 5 – Memory Management & Virtual Memory

PART A – MEMORY MANAGEMENT

1. Basic Concepts of Memory Management

Memory management is a function of the operating system responsible for:

- **Allocation** of memory to processes
- **Protection** and **sharing** of memory
- **Efficient utilization** of main memory
- Providing **abstraction** of memory to processes

Goal: Maximize CPU utilization by minimizing idle memory.

---

2. Logical vs Physical Address Spaces

Logical Address

- Generated by **CPU** during program execution.
- Also called *virtual address*.

Physical Address

- Actual location in **main memory (RAM)**.

Memory Management Unit (MMU)

- Translates logical addresses to physical addresses.
- Enables processes to believe they have continuous memory.

---

3. Memory Allocation

3.1 Contiguous Memory Allocation

Memory is allocated in one continuous block.

Types:

1. Fixed Partitioning

- Memory divided into fixed-size regions.
- **Advantages:** Simple.
- **Problems:**
  - ✓ Internal fragmentation
  - ✓ Limited flexibility

2. Variable Partitioning

- Memory partitions created dynamically as needed.
- **Problems:**
  - ✓ External fragmentation
  - ✓ Need for *compaction*

---

4. Fragmentation

Internal Fragmentation

- Wasted space **inside** allocated block.

- Occurs in fixed partitions.

External Fragmentation

- Enough total free memory exists, but it is not contiguous.

Compaction

- Combines scattered free spaces into a single block.
- Requires relocation of processes.

---

5. Paging

Paging is a non-contiguous memory allocation technique.

Key Components:

- **Page:** Fixed-size block of logical memory.
- **Frame:** Same-sized block in physical memory.
- **Page Table:** Maps pages to frames.

Advantages:

- Eliminates external fragmentation.
- Efficient memory use.

Hardware Support:

- **TLB (Translation Lookaside Buffer)**
- **Page table register**

Protection & Sharing:

- Page table entries include:
  - Read/Write permissions
  - Sharing flags
  - Valid/Invalid bits

Disadvantages of Paging:

- Increased overhead due to page tables.
- Page table can be large.
- Internal fragmentation (within last page).

PART B – VIRTUAL MEMORY

---

1. Basics of Virtual Memory

Virtual memory allows execution of processes **larger than physical memory**.

Benefits:

- More processes can run (multiprogramming).
- Programs don't have to be fully in memory.
- Efficient memory utilization.

---

2. Hardware and Control Structures

Virtual memory uses:

- **Page tables**
- **TLB**
- **Page-frame mapping**
- **Page fault handlers**

Key Concepts:

Locality of Reference

Programs tend to use a small set of pages frequently:

- **Temporal locality** – recently used data is likely reused
- **Spatial locality** – nearby data is often accessed

This makes virtual memory efficient.

---

3. Page Faults

A page fault occurs when:

- CPU requests a page that is **not in the physical memory**.

Handling Steps:

1. Trap to OS.
2. Locate required page on disk.
3. Bring page into memory.
4. Update page table.
5. Restart instruction.

---

4. Working Set

The **working set** of a process is the set of pages it actively uses within a given time window.

Purpose:

- Helps avoid thrashing.
- Tells OS how many frames each process needs.

---

5. Dirty Page / Dirty Bit

- A **dirty bit** indicates a page has been modified.
- When replacing, dirty pages must be written back to disk.

---

6. Demand Paging

Only loads pages into memory **when needed**.

Advantages:

- Reduces memory usage.
- Faster startup.

Disadvantage:

- Causes page faults initially.
- Possible **thrashing** (excessive page faults).

---

7. Page Replacement Algorithms

Used when physical memory is full and a page must be replaced.

7.1 Optimal Page Replacement

- Replace the page that **will not be used for the longest time**.
- Theoretical best performance.
- Not implementable in practice.

7.2 FIFO (First-In First-Out)

- Replace the **oldest** loaded page.
- Simple, but can suffer from **Belady's anomaly**.

7.3 Second Chance (Clock Algorithm)

- Modification of FIFO.
- Uses a **reference bit**.
- Gives pages a "second chance" if recently used.

7.4 NRU (Not Recently Used)

Classifies pages into 4 categories using:

- Reference bit
- Dirty bit

Replaces a random page from the lowest class.

7.5 LRU (Least Recently Used)

- Replaces the page **not used for the longest time**.
- Good performance.
- Requires hardware counters or stack.

Summary of Algorithms

Algorithm	Performance	Hardware Required	Notes
Optimal	Best	Impossible	Used as benchmark
FIFO	Poor/Medium	None	Simple
Second Chance	Good	Reference bit	Practical
NRU	Medium	R & M bits	Simple
LRU	Very Good	Timestamp/Stack	Costly

Full Chapter Summary

- Memory management ensures efficient use of RAM.
- Paging avoids external fragmentation.
- Virtual memory lets programs exceed physical memory.
- Demand paging loads pages only when needed.
- Page faults trigger disk loading of missing pages.
- Page replacement algorithms determine which page to evict.
- LRU and Optimal yield best results; FIFO is simplest.

Chapter 6 – I/O Systems, File & Disk Management

PART A – I/O SYSTEMS

1. I/O Hardware

I/O (Input/Output) devices allow communication between the computer and the external world.

Types of I/O Devices

- **Input:** Keyboard, mouse, scanner
- **Output:** Monitor, printer
- **Storage:** Disk, SSD
- **Communication:** Network cards

Device Controllers

A device controller is hardware that connects a device to the system bus.

Functions:

- Controls device operations
- Contains device-specific logic
- Provides **data buffer**
- Communicates with CPU via **interrupts**

Direct Memory Access (DMA)

Used for high-speed data transfer.

Without DMA:

Device → CPU → Memory (slow)

With DMA:
Device → Memory directly

- Only one interrupt per data block.
- CPU is free to execute other tasks.

---

2. Principles of I/O Software

I/O software manages communication between applications and I/O hardware.

Goals of I/O Software

- Device independence
- Uniform naming of files/devices
- Error handling
- Buffering and caching
- Synchronization

---

3. Interrupt Handlers

What are Interrupts?

Signals from hardware notifying CPU of an event.

Interrupt Handler Responsibilities

- Save CPU state
- Identify interrupt source
- Execute the appropriate handler
- Restart interrupted process

Interrupts improve CPU utilization by avoiding busy waiting.

---

4. Device Drivers

Device drivers are OS modules controlling specific devices.

Functions

- Translate OS requests into device operations
- Handle device-specific commands
- Perform error detection and correction
- Manage buffering

Drivers operate in **kernel mode**.

---

5. Device-Independent I/O Software

Provides general I/O logic independent of specific hardware.

Responsibilities

- Naming and protection
- Device allocation and deallocation
- Buffer management
- Error reporting

PART B – FILE MANAGEMENT

1. Concept of a File

A **file** is a collection of related data defined by its creator.

Attributes of a File

- Name
- Type
- Size
- Location
- Protection (permissions)
- Timestamps

2. File Types

- **Text files**
- **Binary files**
- **Executable files**
- **Source code files**

3. File Access Methods

1. Sequential Access

Read/write data sequentially.
Common for text files.

2. Direct (Random) Access

Access data at any offset.
Used by databases and OS files.

3. Indexed Access

Uses index for faster search.

4. File Operations

- Create
- Open
- Read
- Write
- Reposition (seek)
- Delete
- Close

5. Directory Structure

Directories contain metadata about files.

Common Directory Structures

- **Single level**
- **Two-level**
- **Tree-structured**
- **Acyclic graph**
- **General graph**

---

6. File System Structure

A file system consists of multiple layers:

1. **Application layer**
2. **Logical file system** – metadata management
3. **File organization module** – mapping logical to physical
4. **Basic file system** – issuing commands to devices
5. **I/O control** – device drivers
6. **Hardware layer**

---

7. File Allocation Methods

1. Contiguous Allocation

Files stored in continuous blocks.

Advantages:

- Fast access

Problems:

- External fragmentation
- Difficult to grow files

---

2. Linked Allocation

Each block contains a pointer to the next block.

Advantages:

- No external fragmentation
- Easy file expansion

Problems:

- Slow random access
- Pointer overhead

---

3. Indexed Allocation

Uses an index block to store all block addresses.

Advantages:

- Fast direct access
- No external fragmentation

Problems:

- Index block may become large

---

8. Free-Space Management

Used to track unallocated disk blocks.

Methods

- **Bit Vector (Bitmap)**
  - 0 = free, 1 = allocated
- **Linked List**
- **Grouping**
- **Counting**

---

9. Directory Implementation

- **Linear list**
  - Simple but slow search
- **Hash table**
  - Fast search using hashing

PART C – DISK MANAGEMENT

---

1. Disk Structure

A disk consists of:

- **Platters**
- **Tracks**
- **Sectors**
- **Cylinders**

Disk head moves across cylinders to read/write data.

---

2. Disk Scheduling Algorithms

To reduce seek time and improve throughput.

1. FCFS (First Come First Serve)

Serves requests in arrival order.

2. SSTF (Shortest Seek Time First)

Chooses request closest to current head position.

3. SCAN (Elevator Algorithm)

Head moves in one direction and services requests.

4. C-SCAN (Circular SCAN)

Similar to SCAN but only moves in one direction, jumps to start.

---

3. Disk Reliability

Improving disk reliability through:

- **Redundancy** (RAID)
- **Error-correcting codes (ECC)**
- **Regular backups**

---

4. Disk Formatting

Low-level formatting

- Divides disk into sectors
- Initializes control structures

High-level formatting

- Creates file system structures (superblock, inodes, directory entries)

---

5. Boot Block

Boot block contains the code to start the operating system.

- BIOS loads boot block into memory
- Boot loader loads OS kernel

---

6. Bad Blocks

Disks can develop bad sectors.

Handling Techniques

- Sector sparing
- Remapping
- Marking bad blocks in file system

Chapter Summary

- I/O system consists of devices, controllers, drivers, and interrupts.
- File management handles file creation, access, directories, and allocation.
- Disk management includes scheduling algorithms, formatting, reliability, and handling bad blocks.
- Techniques like DMA, indexed allocation, and SCAN scheduling improve performance.

Parul® University
Vadodara, Gujarat

NAAC GRADE A++

https://paruluniversity.ac.in/